

Dispositivi mobile:

Definizione:

Un dispositivo mobile è un calcolatore elettronico:

- Leggero e portatile.
- Necessita di un sistema operativo ottimizzato per le sue peculiarità.

Caratteristiche principali:

1. Risorse di calcolo limitate:

CPU e RAM meno potenti rispetto ai PC.

Richiedono software ottimizzato e leggero.

2. Input e Output:

Interfacciamento eterogeneo (touch screen, fotocamera, sensori, microfono, ecc.).

Vincolati dalle dimensioni ridotte del dispositivo.

3. Affidabilità delle funzionalità base:

Es. garantire connettività telefonica e servizi minimi essenziali.

4. Energia limitata:

Necessità di ottimizzazione energetica per prolungare la durata della batteria.

Il sistema operativo gestisce sleep mode, risparmio energetico, sospensione app.

5. Eterogeneità:

Supporto a molteplici configurazioni hardware (differenti processori, GPU, modem, sensori).

Richiede un sistema operativo modulare e adattabile.

I dispositivi mobili hanno risorse e vincoli più stretti rispetto ai computer tradizionali, quindi il sistema operativo deve essere ottimizzato in termini di prestazioni, consumo energetico e compatibilità hardware.

Introduzione ad Android:

AOSP – Android Open Source Project:

Android è un sistema operativo open source pensato per i dispositivi mobili (smartphone, tablet, indossabili, ecc.).

Google Inc. è il leader e principale sviluppatore del progetto AOSP.

Target di riferimento:

Dispositivi portatili consumer: Smartphone, Tablet e Wearables (orologi intelligenti, dispositivi fitness, ecc.)

Obiettivo:

Fornire una piattaforma di riferimento comune a tutti i dispositivi portatili, permettendo:

- compatibilità,
- standardizzazione,
- ecosistema condiviso.

Android come progetto Open Source:

Codice e documentazione sono pubblici e accessibili a tutti.

Kernel Linux → rilasciato sotto GNU GPL (General Public License).

Piattaforma Android → rilasciata sotto Apache Software License 2.0 (ASL).

Significa che chiunque può modificare e adattare Android, ma rispettando le licenze:

- GPL → impone la redistribuzione del codice modificato.
- ASL → più permissiva, consente anche uso commerciale con meno vincoli.

Android nasce come progetto aperto e modulare, che Google guida ma che chiunque può contribuire a migliorare o personalizzare.

Caratteristiche di Android:

Basato su Linux:

Android utilizza il kernel Linux per la gestione delle risorse hardware.

Garantisce stabilità, sicurezza e compatibilità con una vasta gamma di dispositivi.

Multitasking e Gestione delle Risorse:

Supporta l'esecuzione simultanea di più applicazioni.

Gestisce CPU, memoria e risorse energetiche in modo efficiente.

Interfaccia Utente Intuitiva:

UI facile da utilizzare e comprensibile.

Adattata ai vari dispositivi di riferimento: Smartphone, Tablet, Smartwatch, TV, Sistemi Automotive.

Sicurezza e Virtualizzazione:

La virtualizzazione isola le applicazioni tra loro e dal sistema.

Garantisce maggiore protezione contro crash o attacchi.

Gestione dell'Energia:

Implementa strategie per ridurre i consumi.

Obiettivo: ottimizzare la durata della batteria .

Framework di Sviluppo:

Fornisce strumenti e API per sviluppare applicazioni native Android.

Supporto a Java, Kotlin, C/C++.

Ecosistema di Applicazioni:

Accesso al Google Play Store.

Possibilità di scaricare milioni di app e giochi, anche di terze parti.

Android è un sistema flessibile, sicuro ed efficiente, con un'ampia diffusione e un ecosistema di app molto ricco.

Cenni storici di Android:

2003: Andy Rubin, Rich Miner, Nick Sears e Chris White fondano Android Inc.

2005: Google acquisisce Android Inc.

2007: Nascita della Open Handset Alliance (84 membri, tra cui Google, produttori HW/SW e compagnie di telecomunicazioni) → diffusione standard aperti nel mobile.

Rilascio con licenza Apache della prima versione Android Beta SDK.

2008: Prima versione ufficiale: Android 1.0, disponibile solo su HTC G1.

2009: Android 1.5 Cupcake → prima versione commerciale, installata su più dispositivi.

2011–2012: Android 4.0 Ice Cream Sandwich → unifica smartphone e tablet con un'esperienza utente coerente.

2014: Android 4.4W KitKat per smartwatch.

Android 5.0 Lollipop → introduce il Material Design (nuovo linguaggio grafico).

2016: Android 7.0 Nougat → introduce ART (Android Runtime) al posto di Dalvik, migliorando prestazioni ed efficienza.

2017+: Rilasci successivi: Android 8.0 Oreo, 9.0 Pie e Android 10, con nuove funzionalità e ottimizzazioni.

Continui aggiornamenti fino alle versioni più recenti.

In sintesi: da progetto startup (2003) → acquisizione Google (2005) → standard mondiale aperto (dal 2007 in poi), Android è diventato il sistema operativo mobile più diffuso al mondo.

Architettura di Android:

1. Linux Kernel (base):

Fornisce l'astrazione dall'hardware.

Gestisce:

- Driver (Audio, Display, Camera, Bluetooth, USB, Wi-Fi, Power Management...)
- Binder (IPC): meccanismo per la comunicazione inter-processo.
- Gestione memoria e risorse

2. HAL (Hardware Abstraction Layer):

Interfaccia tra hardware e sistema operativo.

Permette alle app e al framework di interagire con l'hardware senza conoscerne i dettagli.

Moduli tipici: Audio, Bluetooth, Camera, DRM, Input, Media, Sensors, TV.

3. Native Libraries:

Librerie C/C++ che offrono servizi di base:

- OpenGL/ES (grafica 2D/3D)
- SQLite (database embedded)
- WebKit (motore browser)
- SSL, libc, freetype, media, Surface Manager...

4. Android Runtime (ART):

Sostituisce Dalvik (dalla versione Android 5.0).

Funzionalità:

- Compilazione Ahead-Of-Time (AOT) → maggiore performance.
- Core Libraries: implementano le API base di Java.

5. Framework:

API di alto livello per gli sviluppatori, scritte in Java.

Moduli principali:

- Activity Manager (gestione ciclo di vita app)
- Content Providers (condivisione dati tra app)
- Location Manager, Notification Manager, Package Manager, Window Manager, Resource Manager, Telephony Manager.

6. Apps (livello utente):

Applicazioni native Android (es. Browser, Calendario, Contatti, Media Player, Fotocamera, Messaggi, Email...).

Possono essere di sistema o di terze parti installabili dall'utente.

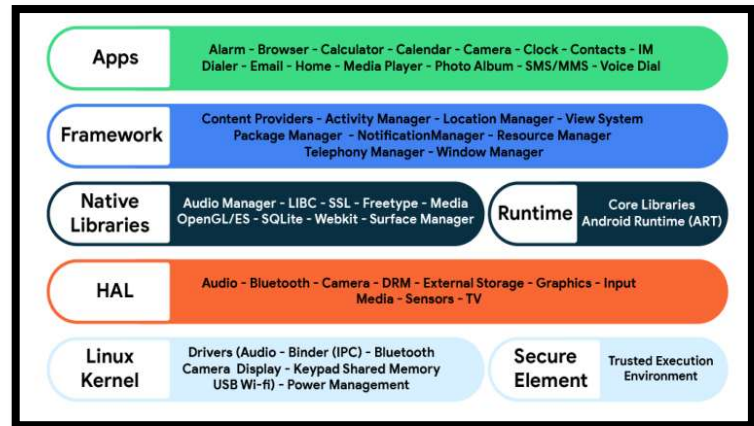
Secure Element:

Modulo dedicato per la sicurezza, es. Trusted Execution Environment (TEE).

Protegge dati sensibili (pagamenti, autenticazioni biometriche, chiavi crittografiche).

In sintesi:

- Kernel = hardware → driver
- HAL = interfaccia hardware
- Native Libs + ART = base software (grafica, database, librerie di sistema)
- Framework = API per sviluppatori
- Apps = livello visibile all'utente



Kernel Linux:

Android si basa su Linux ma aggiunge strumenti specifici per memoria (LMK), energia (wakelocks), comunicazione tra processi (Binder IPC) e storage flash (file system ottimizzati):

Modifiche principali al kernel:

1. Low Memory Killer (LMK) → gestione della memoria aggressiva.
2. Wakelocks → gestione avanzata dell'energia.
3. Binder IPC Driver → meccanismo di comunicazione tra processi.
4. Logcat → servizio di logging centralizzato.

Secure Element → componente hardware che memorizza informazioni sensibili (es. pagamenti NFC, carte elettroniche).

Workflow comune con Linux → Android e Linux condividono lo stesso processo di sviluppo per driver e moduli kernel.

Android eredita il kernel Linux ma lo adatta per memoria, energia, IPC e sicurezza, mantenendo compatibilità nello sviluppo driver.

Low Memory Killer (LMK):

Gestione aggressiva della memoria basata sulla pressione di memoria (vmpressure).

lmkd (Low Memory Killer Daemon):

- misura i ritardi dovuti alla mancanza di memoria;
- più ritardi → più pressione → più probabile killare processi.

vmpressure decide quali task eliminare e quando, per ridurre la pressione.

Gestione dell'energia (wakelocks):

Le app devono richiedere un wakelock per mantenere attive risorse hardware (CPU, schermo, tastiera).

Senza wakelock, Android spegne automaticamente le risorse → risparmio energetico.

Integrato con i servizi di power management di Linux.

Tipi di wakelocks:

1. PARTIAL_WAKE_LOCK → CPU attiva, schermo e tastiera spenti.
2. SCREEN_DIM_WAKE_LOCK → CPU + schermo acceso ma attenuato, tastiera spenta.
3. SCREEN_BRIGHT_WAKE_LOCK → CPU + schermo acceso luminoso, tastiera spenta.
4. FULL_WAKE_LOCK → CPU + schermo acceso luminoso + tastiera accesa.

LMK → evita crash per mancanza di memoria killando processi in base alla pressione.

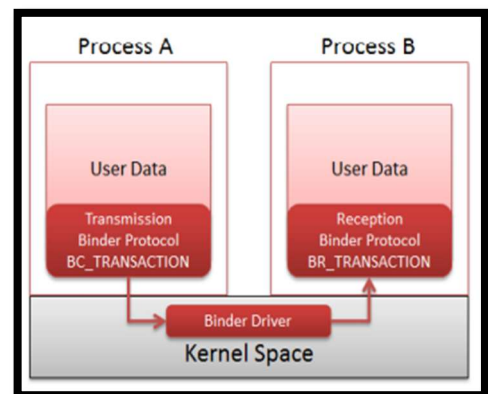
Wakelocks → permettono alle app di mantenere attive risorse, ma in loro assenza Android spegne tutto per risparmiare energia.

Binder IPC Driver → OpenBinder:

È il meccanismo di IPC (Inter Process Communication) usato da Android.

Caratteristiche:

- Sincorno → il processo che invia aspetta la risposta.
- Funziona come canale di comunicazione comune → permette di condividere oggetti tra processi.
- Gestione automatica di allocazione/deallocazione con Reference Counting (contatore di riferimenti agli oggetti).
- Ogni processo ha associato un pool di threads che si occupa della comunicazione.



Il Binder è ciò che consente alle app Android (processi separati) di comunicare in modo sicuro, sincronizzato ed efficiente, senza condividere direttamente memoria.

Dispositivi di storage nei device mobili → Flash Memory:

No seek time → niente latenza da movimento meccanico → accessi casuali molto veloci.

Block erasing → per riscrivere un blocco deve prima essere cancellato (operazione lenta, fatta negli intervalli di inattività).

Wear leveling → ogni cella ha un numero limitato di scritture → occorre distribuire letture/scritture per evitare il deterioramento prematuro.

Requisiti di un file system per dispositivi mobili:

Robusto → supporto a journaling, gestione wear leveling e bad blocks.

Garbage collection → pulizia/compattazione dei blocchi inattivi quando il device è idle.

Portabile → compatibile con vari tipi di dispositivi.

File system Android (ereditati da Linux):

Struttura directory tree (POSIX compliant).

Supporto a più file system: ext4, f2fs, YAFFS2

Android usa memorie flash (veloci ma con limiti di scrittura) → servono file system robusti con wear leveling, garbage collection e journaling, mantenendo compatibilità POSIX.

Hardware Abstraction Layer (HAL):

Cos'è la HAL:

È un livello che definisce un'interfaccia standard tra:

- driver di basso livello (es. audio, bluetooth, camera, storage...)
- servizi di alto livello (Android framework, app, ecc.).

Vantaggi della HAL:

- Permette di implementare nuove funzionalità senza modificare i livelli superiori.
- Il vendor (produttore hardware) definisce la propria HAL usando il linguaggio HIDL (*HAL Interface Definition Language*).
- Grazie alla HAL, Android diventa agnostico rispetto all'hardware → può girare su dispositivi diversi senza modificare il sistema.
- Agevola gli aggiornamenti di Android → i livelli superiori restano invariati anche se cambia l'hardware.

La HAL è il “traduttore” che separa hardware e software: Android non deve sapere i dettagli dei driver, ma usa interfacce standard.

Librerie native:

Offrono funzionalità di basso livello, cuore del sistema:

- grafica
- rendering pagine web
- decoding/encoding dati multimediali

1. Bionic: implementazione della libreria standard C (derivata da BSD).

- Occupa meno spazio della GNU C Library.
- Ottimizzata per dispositivi embedded.

Le app non accedono direttamente alle librerie → passano tramite l'Application Framework.

2. SQLite → DBMS relazionale leggero, portabile e indipendente dalla piattaforma.

3. Surface Manager → gestisce l'interfaccia grafica.

4. SGL e OpenGL/ES → grafica 2D e 3D.

5. Media Frameworks → codec e funzioni per la riproduzione multimediale (basato su OpenCore e Packet Video).

6. FreeType → libreria C per rasterizzare i caratteri.

7. WebKit/Blink → motore di rendering web (usato da browser come Opera e Chrome).

Le Native Libraries sono i “mattoni” del sistema Android → forniscono funzioni essenziali (grafica, media, database, web) ottimizzate per dispositivi mobili.

Il cuore è Bionic, la C library alleggerita per Android.

Android Runtime:

È il Run-time system per le app Android.

Esegue i file in formato .dex (Dalvik Executable) e .oat (Ottimizzati).

Usa una VM register-based (diversa dalla JVM che è stack-based):

Occupa più memoria ma richiede meno istruzioni → maggiore efficienza.

Le app Android sono pacchettizzate in .apk, che contiene:

- codice .dex
- risorse varie (immagini, XML, librerie, ecc.).

.dex (Dalvik EXecutable):

Codice interpretato, indipendente dalla macchina → portabilità (ma performance minori).

Disponibile subito dopo l'installazione.

Hot code → compilato Just-In-Time (JIT).

Cold code → solo interpretato.

.oat (Ottimizzato):

Codice binario eseguibile direttamente.

Generato dal compilatore dex2oat (Ahead-of-Time → AOT).

Vantaggi: tempi di esecuzione minori.

Svantaggi: occupa più spazio (serve memorizzare i binari).

Funzionamento:

Se l'app ha già .oat → ART esegue direttamente i binari.

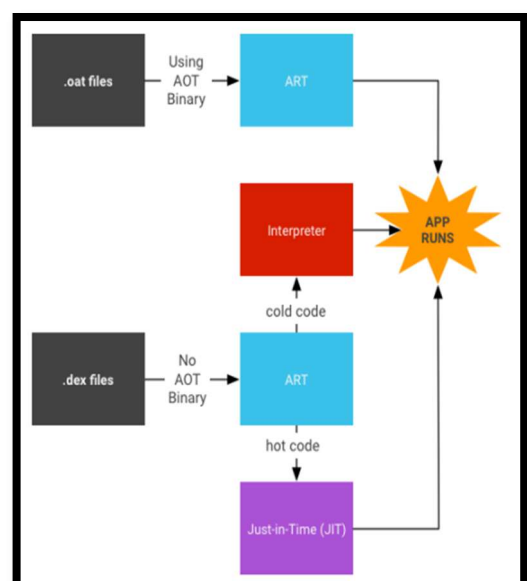
Se l'app ha solo .dex:

- Cold code → interpretato.
- Hot code → compilato JIT (Just-In-Time) durante l'esecuzione.

Android usa .dex per portabilità e compattezza.

Con ART, il codice viene trasformato in .oat (binari AOT) per velocizzare l'avvio e l'esecuzione.

Il runtime bilancia tra interpretazione, JIT e AOT per ottimizzare memoria, performance e compatibilità.



Application Framework:

È l'insieme di librerie e software che forniscono ambiente e servizi per l'esecuzione delle app Android (SDK).

Le API di alto livello richiamano funzioni native per eseguire le operazioni.

API principali:

Resource Manager → accesso a risorse non codice (stringhe, grafici, layout, immagini).

Notification Manager → gestione alert personalizzati (icone, LED, suoni, vibrazioni).

Activity Manager → gestisce le Activity (schermate) e il ciclo di vita.

Telephony Manager → interazione con funzioni del telefono (chiamate, SMS, stato rete).

Location Manager → API per geolocalizzazione e servizi location-based.

Window Manager → gestione finestre delle app sullo schermo.

Package Manager → installazione/rimozione app.

Content Providers → permettono ad un'app di accedere ai dati di altre (es. rubrica).

View System → gestione degli elementi grafici UI (bottoni, griglie, text boxes).

L'Application Framework è lo strato che espone agli sviluppatori le funzionalità del sistema tramite API → semplifica lo sviluppo, gestendo risorse, grafica, notifiche, localizzazione, dati e interfaccia utente.

Applicazioni Android:

Linguaggi principali: Kotlin e Java.

Le app compilate vengono impacchettate con il tool aapt in un file .apk (Android Package), installabile sui dispositivi.

Caratteristiche delle applicazioni Android:

1. Ogni app = singolo processo Linux:
 - Ha la propria istanza di ART (Android Runtime).
 - Ha un unico Linux User ID.
2. Condivisione funzionalità:
 - Ogni app può pubblicare le proprie capabilities e altre app possono riutilizzarle.
3. Architettura flessibile:
 - Le app Android non hanno un singolo entry point → non c'è un "main" unico, ma più componenti attivabili (Activity, Service, BroadcastReceiver, ContentProvider).

Le applicazioni Android sono pacchetti .apk, scritti in Java/Kotlin, eseguiti su ART, isolati come processi Linux ma con possibilità di interazione tramite componenti e capabilities.

Start-up:

All'avvio il kernel esegue lo script init, che inizializza:

- Demoni di sistema (lmkd, usbd, rild...).
- Runtime process → avvia il *service manager* (gestione registrazione e lookup dei servizi).
- Servizio Zygote.

Zygote:

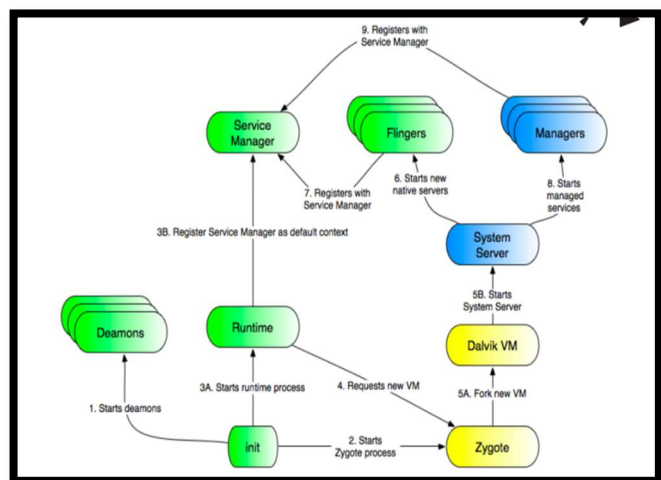
È il processo padre di tutte le istanze di ART dove girano le app.

Funziona così:

- Rimane in ascolto → quando serve una nuova istanza ART, esegue un fork.
- Precarica librerie Java condivise → riduce i tempi di avvio.
- Usa il meccanismo copy-on-write (le risorse sono duplicate solo se modificate).
- Avvia il System Server → gestisce i servizi di base di Android.

Funzionamento:

1. init → avvia i deamons.
2. init → avvia Zygote.
3A. init → avvia il runtime process.
3B. Service Manager registrato come contesto di default.
3. Zygote → fornisce nuove VM su richiesta.
5A. Dalvik/ART VM forkata.
5B. Avvio di System Server.
4. System Server → avvia i native servers (*Flingers*).
5. I server si registrano al Service Manager.
6. Avvio dei Managed Services (*Managers* → Activity, Window, Notification, ecc.).
7. Tutti i servizi sono registrati e disponibili per le applicazioni.



Il boot di Android segue questa sequenza:

Kernel → init → Zygote → ART/Runtime → System Server → Managers & Services → App

Interazione tra livelli:

Le API di alto livello fanno chiamate a funzioni native.

Flussi possibili:

1. App → Runtime Service → Libreria
2. App → Runtime Service → Native Service → Libreria
3. App → Runtime Service → Native Daemon → Libreria

Collegamento tra Application Framework e librerie native tramite JNI (Java Native Interface).

Le componenti native comunicano con:

- Binder IPC
- Socket

Esempi: geolocalizzazione, media player, telefonia.

GPS Example:

1. Application → invoca Location Manager Service (Application Framework).
2. Location Manager Service → chiama GpsLocationProvider.
3. GpsLocationProvider (libreria nativa) → carica dinamicamente libgps.so.
4. libgps.so → interagisce con il driver GPS del kernel

Media Player Example:

1. Application → invoca MediaPlayer (Application Framework).
2. MediaPlayer (Framework) → comunica via JNI con la libreria nativa MediaPlayer.
3. MediaPlayer → usa il Media Framework e comunica con AudioFlinger via Binder IPC.
4. AudioFlinger → carica dinamicamente libaudio.so.
5. libaudio.so → comunica col driver audio del kernel.

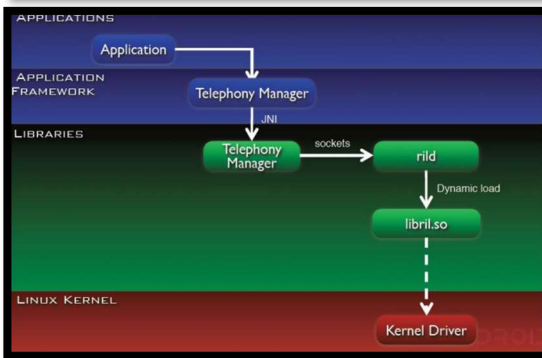
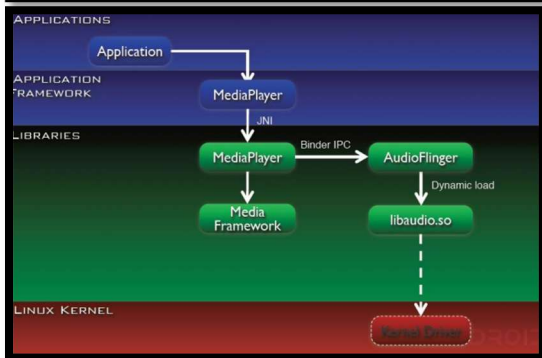
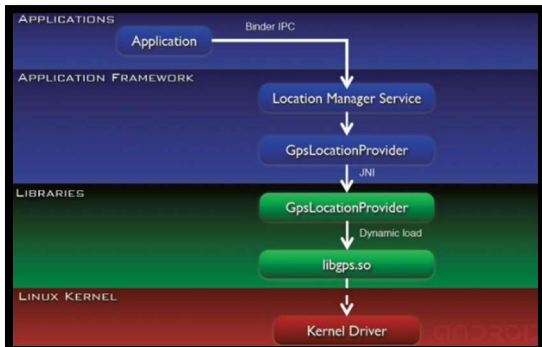
Telephony Example:

1. Application → invoca Telephony Manager (Application Framework).
2. Telephony Manager → via JNI chiama la libreria nativa.
3. La libreria comunica con rilc (Radio Interface Layer daemon) tramite socket.
4. rilc → carica libril.so.
5. libril.so → comunica col driver modem nel kernel.

Recap:

Le app Android non accedono mai direttamente all'hardware → passano sempre da più livelli:

Application → Application Framework → Librerie Native → Kernel Driver con collegamenti tramite JNI, Binder IPC o socket.



Introduzione alle applicazioni:

Cos'è un'applicazione:

Un'app è composta da uno o più componenti.

Ogni componente può essere un entry point per il sistema, l'utente o un'altra applicazione

Tipi di componenti principali:

Activity → rappresenta una schermata con interfaccia utente.

Service → esegue operazioni in background (es. musica).

Broadcast Receiver → risponde a eventi di sistema o app (es. batteria scarica).

Content Provider → condivide dati tra app (es. rubrica).

Fragments → porzioni riutilizzabili di interfaccia.

Intent → meccanismo di comunicazione tra componenti/app.

SDK vs NDK:

SDK (Software Development Kit):

- Sviluppo con Java/Kotlin.
- Le app girano sulla ART VM.
- Sono indipendenti dall'hardware.

NDK (Native Development Kit):

- Permette di scrivere codice nativo in C/C++.
- Usato per scenari specifici (es. videogiochi, elaborazione audio/video).
- Richiede ricompilazione per ogni architettura hardware supportata.

Un'app Android è modulare (diversi entry point) e può essere sviluppata con SDK (Java/Kotlin, portabile) o NDK (C/C++, più vicino all'hardware ma meno portabile).

Activity:

Rappresenta una singola schermata interattiva di un'applicazione (entry point per l'utente).

Caratteristiche:

Ogni Activity è indipendente.

Di solito c'è un'Activity principale (MainActivity) da cui parte l'app.

Un'app può avere più Activity.

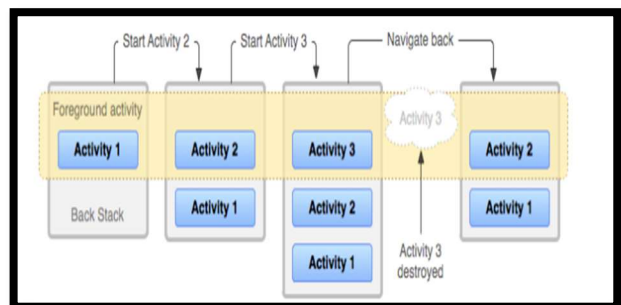
Activity Stack:

Android gestisce le Activity come uno stack (pila LIFO):

- La Foreground Activity è quella visibile all'utente.
- Le altre rimangono nello Back Stack (in background).
- Navigando indietro (back button), si torna all'Activity precedente nello stack.
- Quando un'Activity viene chiusa/distrutta, viene rimossa dallo stack.

Esempio dal grafico:

1. Avvio app → Activity 1 in foreground.
2. Start Activity 2 → Activity 1 va in background, Activity 2 in foreground.
3. Start Activity 3 → Activity 2 in background, Activity 3 in foreground.
4. Navigate back → Activity 3 viene distrutta → ritorno ad Activity 2.



Un'Activity è una schermata → Android le gestisce tramite un Activity Stack, che mantiene l'ordine e consente la navigazione.

Lifecycle Activity:

Un'Activity può trovarsi in tre stati principali:

1. Attiva / in esecuzione → visibile e in primo piano.
2. In pausa → oscurata parzialmente (es. da un dialog).
3. Terminata → chiusa o sostituita da un'altra activity.

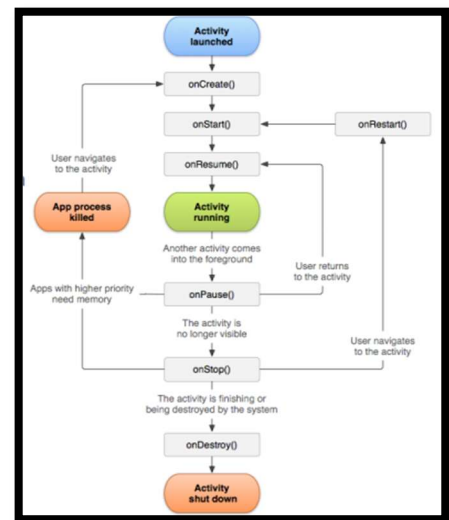
Due concetti importanti:

Visible lifetime → da onStart() a onStop().

Foreground lifetime → da onResume() a onPause().

Metodi principali:

- onCreate() → chiamato alla creazione dell'Activity (setup iniziale).
- onStart() → Activity resa visibile, ma non ancora interattiva.
- onResume() → Activity in foreground, pronta all'interazione con l'utente.
- onPause() → Activity parzialmente oscurata (un'altra sta prendendo il focus).
- onStop() → Activity non visibile.
- onDestroy() → Activity chiusa, risorse deallocate.



Stati killabili: onPause(), onStop(), onDestroy() → se il sistema ha bisogno di memoria, può terminare l'Activity.

Il ciclo di vita di un'Activity definisce come Android gestisce creazione, visibilità, interazione e distruzione delle schermate.

Fragments:

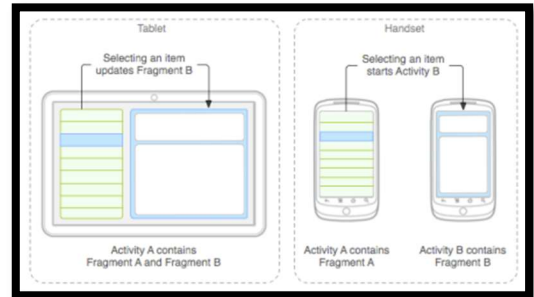
Un Fragment è una componente modulare dell'interfaccia utente di un'app.

Vantaggi principali:

- Favorisce la riutilizzabilità dell'interfaccia e delle sue componenti.
- Consente di creare layout multi-panel adattabili a diversi dispositivi (es. tablet vs smartphone).
- Riduce la duplicazione del codice.

Esempio nell'immagine:

- Tablet → Activity contiene *Fragment A* e *Fragment B* affiancati.
- Smartphone → stessa Activity mostra solo un fragment alla volta.



Lifecycle dei Fragments:

Metodi ereditati da Activity:

`onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, `onDestroy()`.

Metodi specifici dei Fragments:

`onAttach()` → Fragment collegato alla sua Activity.

`onCreateView()` → chiamato per creare la UI del fragment (tra `onCreate()` e `onActivityCreated()`).

`onActivityCreated()` → eseguito quando la Activity che ospita il fragment ha completato la creazione.

I Fragments permettono di scomporre un'app in parti più piccole, riutilizzabili e adattabili a schermi diversi → utili per applicazioni responsive (tablet/smartphone).

Service:

È una componente senza interfaccia grafica che viene eseguita in background.

Esempio:

- Riprodurre musica mentre l'utente usa altre app.
- Recuperare dati in rete senza bloccare l'interazione con un'Activity.

Ciclo di vita dei Service:

Un Service, come un'Activity, ha dei metodi per monitorare i cambiamenti di stato:

- onCreate() → chiamato alla creazione del Service.
- onDestroy() → chiamato quando il Service viene terminato.
- onStartCommand() → invocato quando un'operazione deve essere eseguita in background (tipico dei *Started Services*).
- onBind() → invocato quando un'app si connette al Service per richiedere funzionalità (tipico dei *Bound Services*).

Un Service serve a gestire operazioni in background senza interfaccia utente → può essere avviato (esegue compiti finché non viene fermato) o vincolato ad altre componenti (fornisce funzioni a un'app tramite binding).

Broadcast Receiver:

È una componente passiva che rimane in ascolto di eventi.

Gli eventi possono essere:

- generati dal sistema (es. batteria scarica, arrivo di un SMS, cambio connessione).
- generati da altre applicazioni.

Caratteristiche:

Non è necessario che l'app sia in esecuzione per ricevere l'evento.

Ha un unico metodo di callback: onReceive() → eseguito quando arriva un broadcast.

Esempio pratico:

Il sistema invia un broadcast "BATTERY_LOW".

Tutte le app che hanno registrato un BroadcastReceiver per quell'evento ricevono la chiamata al metodo onReceive().

Il Broadcast Receiver permette di rispondere ad eventi globali senza che l'app sia sempre attiva → è un meccanismo di pub/sub (publish-subscribe) in Android.

Content Provider:

È una componente che gestisce l'accesso ai dati di un'applicazione.

Può essere usato per esporre i dati anche all'esterno dell'app.

Accesso ai dati:

Funziona come un database relazionale.

I dati sono restituiti sotto forma di tabelle.

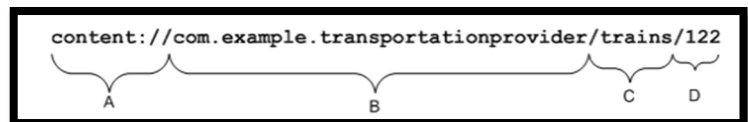
Si accede alle singole istanze tramite un cursore.

Metodi principali da implementare:

- onCreate() → inizializza il Content Provider.
- update() → aggiorna un'istanza.
- remove() → rimuove un'istanza.
- insert() → inserisce una nuova istanza.
- query() → esegue una query e restituisce i risultati.

Struttura:

Ogni Content Provider è identificato da un URI con struttura:



A → protocollo (content)

B → package del content provider

C → tabella

D → istanza

ContentResolver:

Oggetto comune a tutte le applicazioni Android che consente di accedere ai Content Provider specificando l'URI.

Permette di invocare i metodi: update(), remove(), insert(), query().

I Content Provider rendono i dati condivisibili tra applicazioni, con un accesso standard simile a un database SQL, e sono identificati da un URI univoco.

Intent:

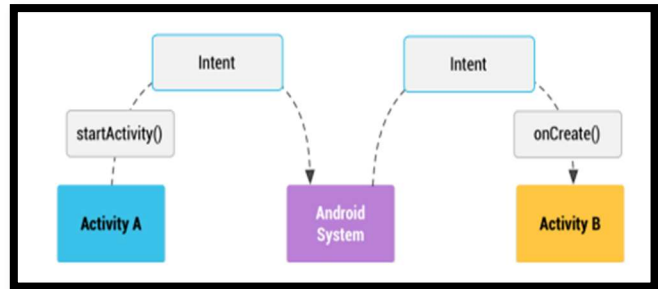
Un Intent è un oggetto generico che rappresenta un'operazione da compiere tra più componenti Android.

Esempi di utilizzo:

- Lanciare una Activity.
- Inviare dati a un Broadcast Receiver.
- Avviare un Service.
- Aprire una pagina web.

Contiene due elementi chiave:

- Azione (cosa fare).
- Dati da elaborare.



Azioni e Dati:

Esempi di coppie azione/dati:

- ACTION_DIAL tel:123 → apre tastierino con numero 123.
- ACTION_VIEW http://www.google.com → apre Google nel browser.
- ACTION_EDIT content://contacts/people/2 → modifica contatto con ID=2.
- ACTION_VIEW content://contacts/people/2 → mostra informazioni di un contatto.
- ACTION_VIEW content://contacts/people/ → mostra lista contatti.

Tipi di Intent:

Esplícito → indica esattamente il destinatario (es. aprire un'Activity specifica della propria app).

Implicito → descrive un compito generico (es. “aprire una pagina web”), sarà Android a scegliere l'app adatta.

Per gestire Intent impliciti → si usa un Intent Filter.

Gli Intent sono il collante della comunicazione tra i componenti Android, permettendo di lanciare activity, servizi, receiver o altre azioni.

Bundle:

È un contenitore di dati chiave-valore.

Usato per passare dati tra Activity (ma anche tra Fragment o Service).

Si può allegare ad un Intent come *extra*.

Usi principali:

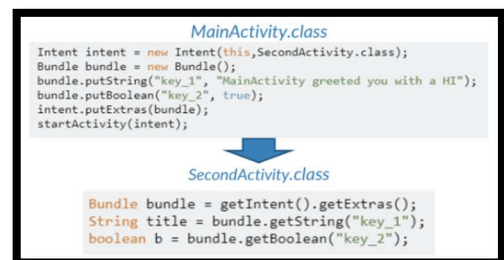
Trasferire dati tra Activity.

Salvare dati di un'Activity prima che venga terminata (es. cambio orientamento schermo).

Esempio di utilizzo:

Si crea un Bundle, si inseriscono dati (putString, putBoolean, ...) e lo si allega all'Intent.

Si recupera il Bundle dall'Intent e si leggono i dati con le chiavi.



Il Bundle è il “pacco dati” che viaggia insieme a un Intent, utile per passare informazioni tra componenti o salvare lo stato di un'Activity.

Android Manifest:

È un documento XML che descrive tutte le informazioni essenziali di un'app Android.

Contiene:

- Package e ID dell'applicazione → identificano univocamente l'app.
- Componenti app → dichiara *Activity*, *Service*, *Broadcast Receiver*, *Content Provider*.
- Intent filters → specificano quali Intent può gestire un componente (es. quale Activity avviare come principale).
- Icone e label → definiscono l'aspetto dell'app (nome e icona).
- Permessi → richiesti per accedere a funzionalità sensibili (es. Internet, Fotocamera, GPS).
- Compatibilità → versione minima e target di Android richieste.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0"
    package="com.example.myapplication">
    <uses-sdk android:minSdkVersion="15" android:targetSdkVersion="26" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".DisplayMessageActivity"
            android:parentActivityName=".MainActivity" />
    </application>
</manifest>
```

L'AndroidManifest.xml è la carta d'identità dell'app, obbligatoria, che definisce chi sei (package/ID), cosa puoi fare (componenti e permessi), e come interagisci con il sistema (intent filter, compatibilità, icona e label).

Sicurezza in Android:

Android applica una serie di meccanismi per proteggere sia il sistema operativo che i dati dell'utente.

Principi chiave:

1. Sandbox:

Ogni applicazione è confinata in un ambiente isolato chiamato *sandbox*.

Le app non possono accedere direttamente ai dati o ai file di altre app.

Questo isolamento avviene tramite UID univoco (Unique User Identifier): ogni app gira come un utente Linux separato.

2. Permessi:

Lo sviluppatore deve dichiarare esplicitamente nel Manifest i permessi necessari.

L'utente deve concedere tali permessi per consentire l'accesso a risorse sensibili:

- SMS
- Rete telefonica
- Lettura/Scrittura file
- Accesso a contatti, posizione GPS, fotocamera, microfono
- Gestione energetica, Internet, ecc.

3. Access Control Policy:

Le richieste di accesso vengono valutate in base a regole di sicurezza e permessi.

Si applicano controlli sia a livello di file system che di Application Database.

Schema della slide:

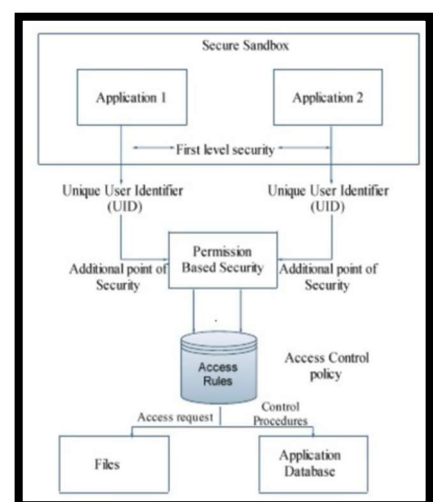
Application 1 e 2 → girano in sandbox separate.

UID univoco → isola ogni app.

Permission Based Security → controlla le richieste di accesso.

Access Rules & Control Procedures → regolano l'uso di file e database delle app

Android garantisce sicurezza tramite sandboxing, permessi espliciti e controllo centralizzato delle risorse.



Permessi:

Tipi di permessi:

1. Predefiniti:

Già forniti da Android.

Servono per accedere a funzionalità standard → es. invio SMS, accesso a Internet, memoria interna, posizione GPS.

Si richiedono con: `<uses-permission android:name="android.permission.RECEIVE_SMS"/>`

2. Custom:

Creati dallo sviluppatore per la propria applicazione.

Servono a proteggere componenti interni (Activity, Service, Content Provider).

Definiti all'interno del Manifest:

```
<permission android:name="com.example.project.PERMISSION" android:protectionLevel="normal"/>
```

Definizione dei permessi custom:

Nel AndroidManifest.xml si possono specificare:

- name → nome univoco del permesso.
- label, description, icon → informazioni testuali/grafiche.
- permissionGroup → gruppo di appartenenza.
- protectionLevel → livello di protezione (es. normal, dangerous, signature)

Esempio:

```
<permission  
    android:name="com.example.myapp.PERMISSION"  
    android:label="Permesso speciale"  
    android:description="Necessario per accedere a funzioni custom"  
    android:protectionLevel="dangerous"/>
```


Gruppi di permessi:

È possibile creare permission-group per raggruppare permessi simili.

Se un'app richiede un gruppo, è come se chiedesse tutti i permessi contenuti in quel gruppo.

Esempio:

```
<permission-group  
    android:name="com.example.group.MY_GROUP"  
    android:label="Gruppo Permessi Personalizzati"  
    android:description="Permessi speciali"/>
```

Data Storage in Android:

Preferenze (SharedPreferences):

Memorizzano variabili semplici (key-value) in un file XML privato.

Metodi:

- `getPreferences()` → preferenze legate ad una singola Activity.
- `getSharedPreferences()` → preferenze dell'intera app.
- `getDefaultSharedPreferences()` → preferenze globali di Android.

Ottimo per salvare configurazioni (es. tema scuro/chiaro, login).

Internal Storage:

Memoria interna del dispositivo.

Richiede permessi espliciti per lettura/scrittura.

I file salvati sono privati all'app, ma possono diventare condivisi.

Scoped Internal Storage:

Introdotta per maggior sicurezza e privacy.

Ogni app ha una cartella dedicata e isolata, accessibile solo da sé stessa.

Evita conflitti tra app.

SQLite:

Android integra un DB relazionale locale.

Implementa gran parte dello standard SQL-92.

Ogni app ha il proprio database privato.

Per accedere dall'esterno serve un Content Provider.

Supporta CRUD completo (create, read, update, delete)

Network Connection:

I dati possono essere salvati su internet.

Android fornisce API (java.net, android.net) per accedere a dati remoti.

Usato per sincronizzazione cloud, API REST, backend remoto.

In sintesi:

Preferenze → dati semplici (key-value).

Internal/Scoped Storage → file locali.

SQLite → database strutturato.

Network → salvataggio remoto.

Risorse:

I Resource sono file esterni usati nel codice delle app Android.

Sono inclusi all'interno del file .apk.

Supportano vari tipi di file: immagini (PNG, JPEG), documenti XML, file multimediali.

Directory principali delle risorse:

Directory	Resource type
res/anim/	File xml che descrivono delle animazioni
res/drawable/	Questa directory deve essere usata per le immagini necessarie al software. Sono supportati i formati PNG e JPEG
res/layout/	File XML per specificare il layout delle interface utente
res/values/	File XML che descrivono stringhe e altri parametri utilizzati dall'applicazione.
res/xml/	Questa directory ospita file XML di tipo qualsiasi, che il sistema interpreta e rende semplici da acquisire
res/raw/	Questa directory ospita file binari di tipo qualsiasi (ad esempio audio o filmati) che vengono copiati nell' .apk e l'applicazione poi carica

In sintesi:

res/drawable → immagini

res/layout → interfacce

res/values → stringhe, stili e parametri

res/raw → file multimediali e binari

res/anim → animazioni

res/xml → configurazioni generiche